



King Abdulaziz University
Faculty of Computer and Information
Technology
Computer Science Department



Recommendation System

Team Members
Turki Khalid Alqahtani
Rayan Yosof Fadhil
Raed Ahmed Alsafry
Omar Waleed Alsiary

Artificial Intelligence 1



Fall 2024

TABLE OF CONTENTS

1.	Introduction	3
1.1	Research Fundamentals of Recommendation System	3
1.2	Development Environment	3
1.3	Core Concepts And Techniques	3
1.3.1	Data collection.....	3
1.3.2	Recommendation Algorithms	4
1.3.3	Machine learning techniques.....	4
1.3.4	Privacy and Ethics	5
1.3.5	Tools and Frameworks	5
1.4	Dataset.....	5
1.5	Approaches And Methods	5
1.6	Modern Algorithms And Models	6
2.	What Is My Data?.....	6
2.1	Data Source.....	6
2.1.1	Origin.....	6
2.1.2	Collection Method	6
2.1.3	Context	7
2.2	Data Type.....	7
2.2.1	Data Types	7
2.3	Missing and Incomplete Data	7
2.3.1	Missing Values	7
2.3.2	Data Completeness	7
2.3.3	Imputation Strategy	7
2.4	Assess Data Distribution	7
2.4.1	Descriptive Statistics.....	7
2.4.2	Visual Exploration.....	8
2.4.3	Data Range.....	10
2.5	Correlations and Relationship.....	10



2.5.1	Dimensionality.....	10
2.5.2	Feature Correlations.....	10
2.6	Data Biases.....	11
3.	Data Cleaning and Preprocessing.....	11
3.1	Duplicates.....	11
3.2	Normalization/Standardization.....	11
3.3	Feature Engineering.....	11
3.4	Analyze Data Quality.....	12
4.	Documentation.....	12
4.1	Summarization of Key Findings.....	12
4.2	Data Quality.....	12
4.3	Feature Selection.....	12
5.	Collaborative filtering (User-based).....	13
5.1	Adding Movies Title.....	13
5.2	Exploratory Data Analysis (EDA).....	14
5.3	Create User-Movie Matrix.....	14
5.4	Data Normalization.....	14
5.5	Identify Similar Users (Pearson Correlation).....	15
5.5.1	Picking User.....	15
5.6	Narrow Down Item Pool.....	16
5.6.1	Step 1.....	16
5.6.2	Step 2.....	16
5.6.3	Step 3.....	16
5.7	Recommend Items.....	17
6.	Popularity-Based Filtering (Extra).....	18
6.1	Overall Popularity.....	18
6.2	Average Rating Popularity.....	18
Figure 1: Histogram of Rating.....		8
Figure 2: Boxplot of Rating.....		9
Figure 3: Distribution of Interactions per User and per movie.....		9
Figure 4: Histogram of Rating. (Smaller).....		11



1. Introduction

1.1 Research Fundamentals of Recommendation System

1. Data collection and management.
2. The type of recommendation approach.
3. Algorithm selection.
4. Personalization and filtering.
5. Evaluation and Metrics.
6. User Interface and Experience.
7. Privacy and Ethics.
8. Scalability and Performance.

1.2 Development Environment

Python is one of the most used languages in the field of Artificial Intelligence, it has a wide set of libraries & frameworks that can ease building an artificial intelligence agent, thus, we have chosen **Python** to be our main development environment in which we can build our Artificial Intelligence project.

1.3 Core Concepts And Techniques

1.3.1 Data collection

The first step in building a recommendation system is gathering data, which typically includes:

- **User information:** collect information about the user and the users' behavior, such as browsing history, purchase history, searches and clicking or checking out a product.
- **Product information:** collect information about the products. Such as description, categories, price range, ratings and reviews.
- **Transaction data:** collect transaction data of past purchases such as items bought, quantity of each item, and purchase timestamps.

1.3.2 Recommendation Algorithms

There are two types of algorithms you can use to generate recommendations, each with its strengths and limitations:

- **content-based filtering:** these types of algorithms recommend products based on attributes of items the user has shown interest in. for example, if the user has bought or liked a product from a specific brand, the system will suggest other products from that brand.
- **Collaborative filtering:** collaborative filtering recommends products based on the preference of similar users. However collaborative filtering has a problem called the cold start problem. When you start your recommendation system but you still don't have ratings from users, you can't use collaborative filtering. the solution to this problem would be to use content-based filtering at first, and once you get more users and user reviews and ratings you can start using collaborative filtering.

Most of the powerful recommendation systems use both types to create a “**hybrid recommendation system**”.

1.3.3 Machine learning techniques

- **Matrix Factorization:** This is a method that uses techniques like Singular Value Decomposition (SVD) or Alternating Least Squares (ALS) to break down data about how users interact with items. This helps find hidden patterns that explain why users might like certain items.
- **Deep Learning:** Neural networks can be used to model complex non-linear relationships between users and items. Techniques like autoencoders, recurrent neural networks, or convolutional neural networks for image-based recommendations are common.

1. Implementation Considerations

- **Scalability:** The system should efficiently handle a large number of users and items.
- **Performance:** Recommendation generation should be quick to ensure a good user experience.



1.3.4 Privacy and Ethics

Guarantee user information is handled with care, obeying regulations like GDPR or CCPA. Provide users with explanations or transparency regarding how recommendations are produced and how their details are utilized.

1.3.5 Tools and Frameworks

- **Python Libraries:** Scikit-learn, TensorFlow, PyTorch, and surprise are popular for building recommendation systems.
- **Platforms:** Apache Mahout, Spark MLlib for scalable machine learning algorithms.

1.4 Dataset

For building an e-commerce recommendation system, we'll need the following datasets:

- **User Behavior:** Contains user interactions (views, ratings, watch history).
 - Columns: user_id, movie_id, event_type, timestamp.
- **Movie Data:** Includes details about the movies like genre, release date, director.
 - Columns: movie_id, name, category, price, rating.
- **Transaction History:** Purchase or rental records for movies.
 - Columns: order_id, user_id, movie_id, purchase_time, quantity.
- **Reviews & Ratings:** User feedback on ,movies.
 - Columns: user_id, movie_id, review, rating.
- **Optional - User Demographics:** Age, gender, location of users.

There are tons of ready-to-use datasets for easing our module, we will use User-Movie dataset available in Grouplens and GitHub.

1.5 Approaches And Methods

1. **Collaborative Filtering:** This approach suggests items by finding patterns in user preferences or behaviors. It can be user-based, recommending items a user might like based on similar users' interests, or item-based, suggesting items similar to those a user has previously liked.
2. **Content-Based Filtering:** This method suggests items by considering the characteristics of the items and the user's preference profile, proposing similar items to those previously liked by the user.

3. **Popularity-Based Recommendation:** This is a straightforward method suggests popular items to all users, irrespective of their preferences. It is commonly employed for new users who have limited or no historical data available.
4. **Rule-Based Recommendation:** This method uses a set of predefined rules to make recommendations. For example, "If a user buys a movie, recommend movie's next chapter".

1.6 Modern Algorithms And Models

1. **Field-aware Factorization Machines (FFM):** FFM extends FM by considering different fields or feature categories independently. In e-commerce, this can be useful when dealing with diverse categorical features like user demographics, item attributes, and user-item interactions, allowing for more granular feature interactions.
2. **GPT (Generative Pre-trained Transformer):** GPT models can generate personalized recommendations by leveraging pre-trained transformer architectures to understand user preferences and generate item suggestions based on historical interactions.
3. **Gradient Boosting Machines (GBM):** GBM can boost recommendation accuracy by combining the predictions of multiple base models. They are effective at capturing complex interactions between features and enhancing the overall recommendation performance.
4. **Neural Collaborative Filtering (NCF):** NCF combines the strengths of neural networks and collaborative filtering, enabling the model to learn intricate patterns from user-item interactions in e-commerce data. This can lead to more accurate and personalized recommendations.

2. What Is My Data?

2.1 Data Source

2.1.1 Origin

User rating, User ID, Movie ID, and timestamp details are among the data derived from user-movie interactions on MovieLens.

2.1.2 Collection Method

The dataset, sourced from [GroupLens](https://grouplens.org/datasets/movielens/), originates from user interaction data on a movie recommendation platform. However, the collection method is not specified.



2.1.3 Context

The data was collected to provide structured information for analysis and machine learning projects. It supports tasks like data mining and AI model training, offering a resource for developers and researchers. We've decided to use this data since every row is a user with their movie. Which makes it easier for us in the recommendation system.

2.2 Data Type

2.2.1 Data Types

For our finalized dataset, we have four columns:

- ❖ User ID: Numerical.
- ❖ Movie ID: Textual.
- ❖ Rating: Numerical (range 0 – 5 Increasing by .5).
- ❖ Time Stamp: Numerical.

2.3 Missing and Incomplete Data

2.3.1 Missing Values

Movie Title is missing.

2.3.2 Data Completeness

No incomplete data, all users in the dataset have rated the items.

2.3.3 Imputation Strategy

There is a dataset containing (Movie Title and Movie ID), we merged it to our train data (Movie ID) as a middleman.

2.4 Assess Data Distribution

we've used the **Pandas** library, which is widely used for data manipulation and analysis in Python. it helped us for giving statistics and plotting of our dataset.

2.4.1 Descriptive Statistics

- **Number of Unique:** We've calculated the number of unique users, movies and ratings.

Number of unique users: 610

Number of unique movies: 9724

Number of unique ratings: 10

- **Rating Statistics:** We've calculated mean, median, variance and standard deviation.

Mean of rating: 3.501

Median of rating 3.5

Variance of rating: 1.087

Standard Deviation of rating: 1.042529

Highest of rating: 5.0

Lowest of rating: 0.5

2.4.2 Visual Exploration

- **Histogram**

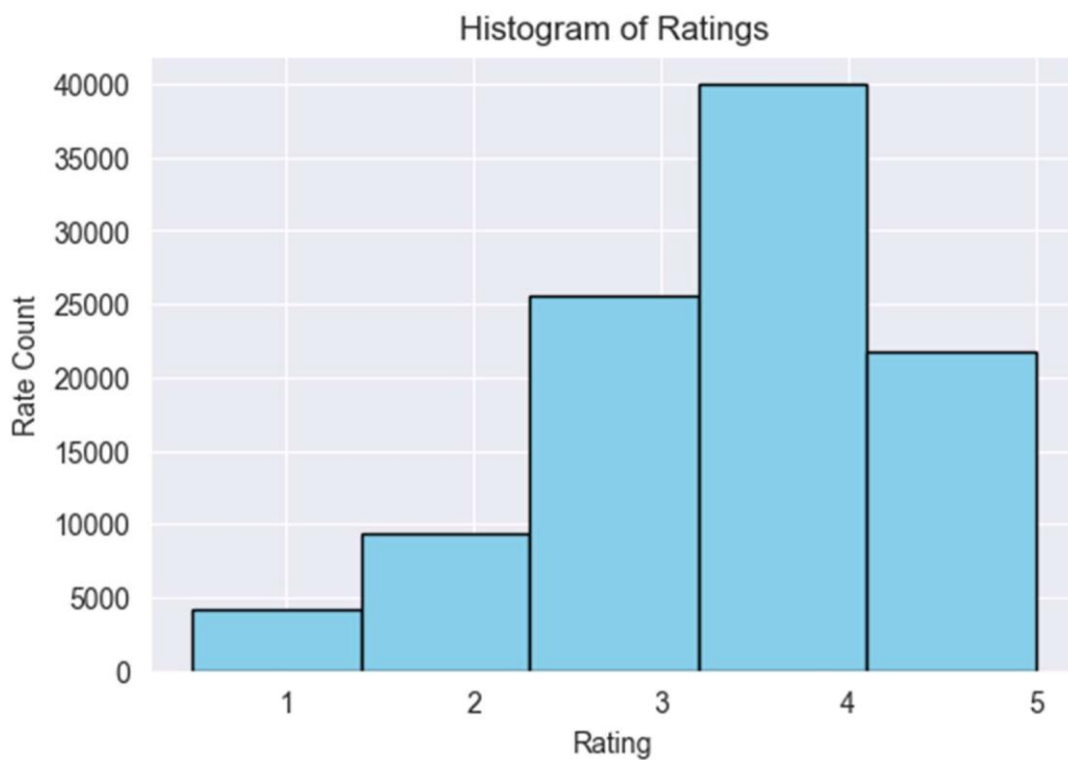


Figure 1: Histogram of Rating.

- **Boxplot**

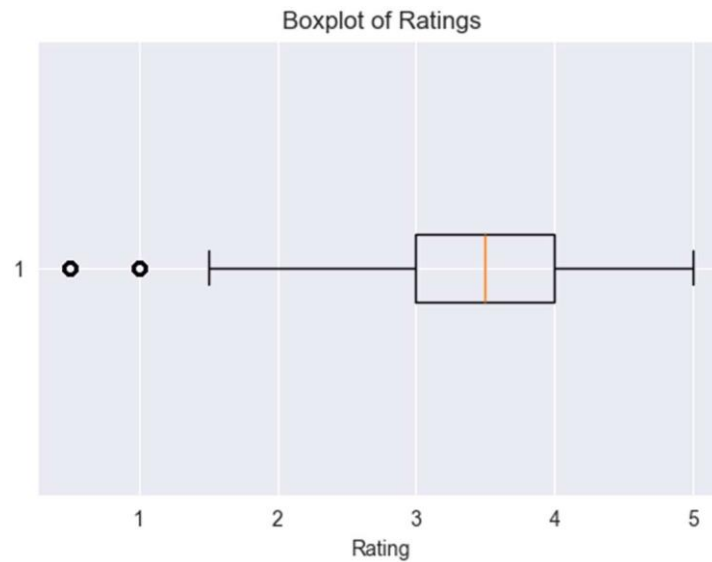


Figure 2: Boxplot of Rating.

- **Distribution of Interactions**

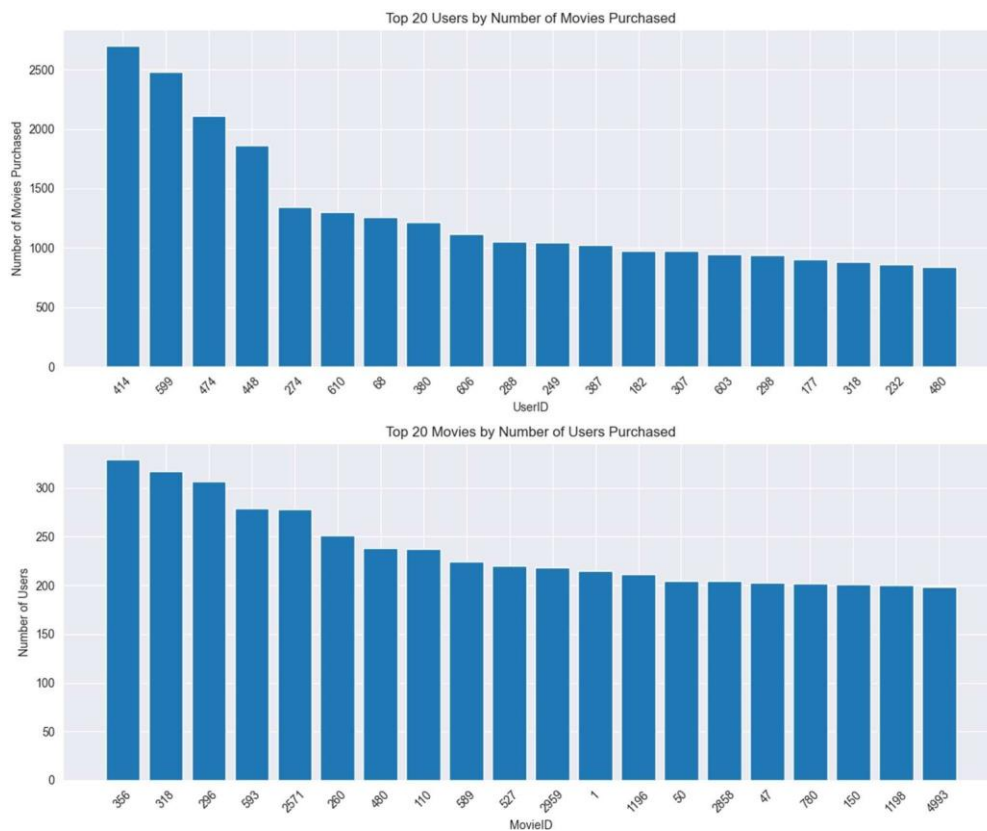


Figure 3: Distribution of Interactions per User and per movie.

2.4.3 Data Range

	Rating
count	100836.0
25%	3.000000
50%	5.000000
75%	5.000000

2.5 Correlations and Relationship

2.5.1 Dimensionality

the tabular dataset of the User-Movies contains 100,836 observations (rows) and 4 features (columns).

2.5.2 Feature Correlations

To make a correlation matrix that will explore relationships between features, we can only use numeric features from the dataset, listed below are the only features applicable to measuring correlation between each other:

- 1- Average Rating.
- 2- Number of Rating (Popularity).

Other columns had unapplicable data to calculate a meaningful correlation. This problem normally comes from many missing values or when the data isn't numeric or consistent across columns.

Here is the correlation matrix for the applicable numeric columns in the dataset:

	Number of Rating	Avg Rating
Number of Rating	1.00000	0.1272585
Avg Rating	0.1272585	1.00000

From the matrix we can see that there is correlation between avg rating and number of ratings. Indicating that if movie gets popular then rating of the movie will raise by 0.127.

To calculate the correlation matrix, I used the `corr()` method from the pandas library, which operates on a DataFrame object.

2.6 Data Biases

Missing values in a dataset can lead to an uncomplete and biased dataset. In this dataset there were no missing or null values, ensuring a complete dataset. In terms of ratings, most ratings are 3.5 and 4 out of 5 with over 40 thousand reviews. While 3-star and 5-star reviews are nearly 20 thousand 1– 2.5 star reviews are less than a 10 thousand, which shows an unbalance in reviews and a bias towards middle.

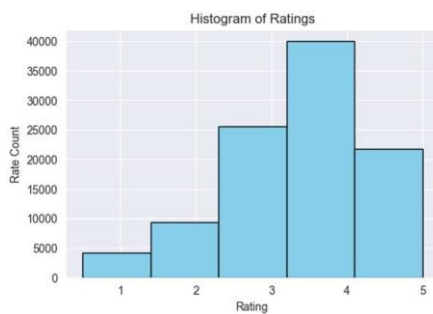


Figure 4: Histogram of Rating. (Smaller)

3. Data Cleaning and Preprocessing

3.1 Duplicates

It is an importing step to check if there any duplicates in the dataset. For that we will use the method `.duplicated().sum()` in our dataset. After checking we got that there were no duplicates in the dataset.

```
train_data.duplicated().sum()
```

0

3.2 Normalization/Standardization

We will normalize when we apply collaborative filtering since some people tend to give a higher rating than others.

3.3 Feature Engineering

A tool that monitors popular products based on review count is something we are thinking about implementing. This tool will assist in determining which goods are currently popular by keeping track of review numbers. Users will be able to quickly find popular products and make better decisions as a result.

3.4 Analyze Data Quality

- **Label Quality:** In order to check for our dataset if it has a good quality, We used the method `.isnull().sum()` to give us the total number of missing value in our dataset. As result we didn't have any missing value.

```
train_data.isnull().sum()
✓ [334] 14ms
```

	123 <unnamed>	
UserID	0	
MovieID	0	
Rating	0	
timestamp	0	

- **Data Consistency:** We improved the consistency in our dataset by merging title movie in columns, so it is easy to map them since we didn't had any title to check in the original file.
- **Data Completeness:** There was no missing data in our dataset.

4. Documentation

4.1 Summarization of Key Findings

Our dataset is ideal for applying collaborative filtering recommendation algorithms, as it contains no anomalies or areas of concern that would negatively impact the performance of such algorithms.

4.2 Data Quality

The dataset consists of only four columns: UserID, MovieID, Rating, Timestamp. We do not have product names or detailed product information. Despite searching on platforms like MovieLens, we found a dataset contains movie titles with their id.

4.3 Feature Selection

We have included collaborative filtering based on user interactions (ratings) as the core recommendation approach. Additionally, we included popular movies based on number of rating and average rating to enhance relevance, but we excluded content-based filtering due to a lack of time, we would love to add it in the future.

5. Collaborative filtering (User-based)

Firstly, let's understand how User-based collaborative filtering works. User-based collaborative filtering makes recommendations based on user-item interactions in the past. The assumption behind the algorithm is that similar users like similar movies.

User-based collaborative filtering algorithm usually has the following steps:

- 1- Adding Movies Title
- 2- Exploratory Data Analysis (EDA)
- 3- Create User-Movie Matrix
- 4- Data Normalization
- 5- Identify Similar Users (Pearson Correlation)
- 6- Narrow Down Item Pool
- 7- Recommend Items

5.1 Adding Movies Title

We will add movies' title to our dataset by reading the file and merging it.

```
# Read data
movies = pd.read_csv('movies.csv')

movies.head()
✓ [345] 22ms
```

movieId	title	genres
1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
2	Jumanji (1995)	Adventure Children Fantasy
3	Grumpier Old Men (1995)	Comedy Romance
4	Waiting to Exhale (1995)	Comedy Drama Romance
5	Father of the Bride Part II (1995)	Comedy

Using 'movieId' as matching key to our dataset 'MovieID'

```
# Merge ratings and movies datasets
mtrain_data = pd.merge(train_data, movies, left_on='MovieID', right_on='movieId', how='left')

# Drop the duplicate column 'movieId' from the merged dataset
mtrain_data.drop(columns=['movieId'], inplace=True)

mtrain_data.head()
✓ [346] 27ms
```

UserID	MovieID	Rating	timestamp	title
1	1	4.000000	964982703	Toy Story (1995)
1	3	4.000000	964981247	Grumpier Old Men (1995)
1	6	4.000000	964982224	Heat (1995)

5.2 Exploratory Data Analysis (EDA)

We have around 100 thousand ratings, and we will create a matrix with 100,000 x 100,000. Thus, our computer can't handle the computation.

Error: **Unable to allocate 30.4 GiB for an array.**

The solution is by decreasing number of movies who have < 100 number of ratings thus every user who rate a movie that is not famous it will get delete. Same as YouTube and other platforms, it only recommends famous videos.

By applying it we get 134 unique movies with 597 unique users with 19,788 rows instead of 100,000 rows.

5.3 Create User-Movie Matrix

We will transform the dataset into a matrix format, the rows of the matrix are users, and the columns of the matrix are movies, the value of the matrix is the user rating of the movie if there is a rating. Otherwise, it shows 'NaN'.

```
# Create User-Product matrix
matrix = train_data_100.pivot_table(index='UserID', columns='title', values='Rating')
matrix.head()
```

✓ [352] 31ms

	2001: A Space Odyssey (1968)	Ace Ventura: Pet Detective (1994)	Aladdin (1992)	Alien (1979)	Aliens (1986)
1	NaN	NaN	NaN	4.000000	NaN
2	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	4.000000	NaN	NaN
5	NaN	3.000000	4.000000	NaN	NaN

5.4 Data Normalization

Since some people tend to give a higher rating than others, we normalize the rating by extracting the average rating of each user.

```
# Normalize user-product matrix
matrix_norm = matrix.subtract(matrix.mean(axis=1), axis='rows')
matrix_norm.head()
```

✓ [353] 29ms

	2001: A Space Odyssey (1968)	Ace Ventura: Pet Detective (1994)	Aladdin (1992)	Alien (1979)	Aliens (1986)	Amelie (Fabuleux destin d'Amélie)
1	NaN	NaN	NaN	-0.392857	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	0.617647	NaN	NaN	NaN
5	NaN	-0.461538	0.538462	NaN	NaN	NaN

After normalization, the movies with a rating less than the user's average rating get a negative value, and the products with a rating more than the user's average rating get a positive value.

5.5 Identify Similar Users (Pearson Correlation)

We will calculate the user similarity matrix using Pearson Correlation.

Pearson Correlation will get Normalized User-Movie matrix and will take each user and compare with all users, if User 1 all watched movies are in User 2 watched movies it will be 1.0 (same taste). Else -1 means opposite movie preference.

The matrix is User x User

```
# User similarity matrix using Pearson correlation
user_similarity = matrix_norm.T.corr()
user_similarity.head()
```

✓ [354] 40ms

UserID	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	
1	1.000000	NaN	NaN	0.391797	0.180151	-0.439941	-0.029894	0.464277	1.000000	-0.037987	0.385758	NaN	NaN	NaN	0.175000	0.305392	0.293103	0.088600
2	NaN	1.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.000000	NaN	NaN	NaN	NaN	-0.752651	0.000000	-0.648886	0.000000
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	0.391797	NaN	NaN	1.000000	-0.394823	0.421927	0.704669	0.055442	NaN	0.360399	0.872872	NaN	NaN	NaN	0.215166	0.408228	0.046849	-0.194802
5	0.180151	NaN	NaN	-0.394823	1.000000	-0.004888	0.328889	0.030168	NaN	-0.777714	0.486854	1.000000	NaN	NaN	0.252039	-0.012067	-0.684653	0.310833

5.5.1 Picking User

Let's say we will pick user 1, so we will drop the row with user 1 since no need for it. Then choosing Number of similar users, and selecting User similarity threshold, lastly, we get the similar users to user 1.

```
# Number of similar users
n = 10

# User similarity threshold
user_similarity_threshold = 0.3

# Get top n similar users
similar_users = user_similarity[user_similarity[picked_userid]>user_similarity_threshold][picked_userid].sort_values(ascending=False)[:n]

# Print out top n similar users
print(f'The similar users for user {picked_userid} are', similar_users)
```

✓ [400] 13ms

```
The similar users for user 1 are UserID
108    1.000000
9      1.000000
550    1.000000
598    1.000000
502    1.000000
401    0.942809
511    0.925820
366    0.872872
154    0.866025
595    0.866025
Name: 1, dtype: float64
```


5.6 Narrow Down Item Pool

5.6.1 Step 1

Remove the movies that targeted user '1' has not been watched.

```
# Movies that the target user has watched
picked_userid_watched = matrix_norm[matrix_norm.index == picked_userid].dropna(axis=1, how='all')
picked_userid_watched
✓ [408] 26ms
```

title	Alien (1979)	American Beauty (1999)	American History X (1998)	Apocalypse Now (1979)
1	-0.392857	0.607143	0.607143	-0.392857

5.6.2 Step 2

In the similar users we need to delete all the movies that all the similar users have not watched.

```
# Movies that similar users watched. Remove movies that none of the similar users have watched
similar_user_movies = matrix_norm[matrix_norm.index.isin(similar_users.index)].dropna(axis=1, how='all')
similar_user_movies
✓ [402] 10ms
```

title	Aladdin (1992)	Alien (1979)	Amelie (Fabuleux destin d'Amélie Poulain, Le) (2001)	Back to the Future (1985)	Batman Begins (2005)
9	NaN	NaN	NaN	NaN	NaN
108	NaN	0.466667	NaN	0.466667	NaN
154	NaN	NaN	NaN	NaN	NaN
366	NaN	NaN	-0.205882	NaN	NaN
401	-0.382353	NaN	NaN	NaN	-0.382353
502	NaN	NaN	NaN	NaN	NaN
511	NaN	0.453846	NaN	NaN	NaN

5.6.3 Step 3

Drop movies in the similar users matrix who watched same movies like targeted user '1'.

```
# Remove the watched movie from the movie list
similar_user_movies.drop(picked_userid_watched.columns, axis=1, inplace=True, errors='ignore')

# Take a look at the data
similar_user_movies
✓ [403] 28ms
```

title	Aladdin (1992)	Amelie (Fabuleux destin d'Amélie Poulain, Le) (2001)	Batman Begins (2005)	Beautiful Mind, A (2001)
9	NaN	NaN	NaN	NaN
108	NaN	0.466667	NaN	0.466667
154	NaN	NaN	NaN	NaN
366	NaN	NaN	-0.205882	NaN

5.7 Recommend Items

We will decide which movie to recommend to the target user. The recommended items are determined by the weighted average of used similarity score and movie rating. The movie rating are weighted by the similarity score, so the users with higher similarity get higher weights.

The code loops through items and users to get the item score, rank the score from high to low and pick the top 10 movies to recommend to targeted user '1'.

```
1 # A dictionary to store item scores
2 item_score = {}
3
4 # Loop through items
5 for i in similar_user_movies.columns:
6     # Get the ratings for movie i
7     movie_rating = similar_user_movies[i]
8     # Create a variable to store the score
9     total = 0
10    # Create a variable to store the number of scores
11    count = 0
12    # Loop through similar users
13    for u in similar_users.index:
14        # If the movie has rating
15        if pd.isna(movie_rating[u]) == False:
16            # Score is the sum of user similarity score multiply by the movie rating
17            score = similar_users[u] * movie_rating[u]
18            # Add the score to the total score for the movie so far
19            total += score
20            # Add 1 to the count
21            count += 1
22    # Get the average score for the item
23    item_score[i] = total / count
24
25 # Convert dictionary to pandas dataframe
26 item_score = pd.DataFrame(item_score.items(), columns=['movie', 'movie_score'])
27
28 # Sort the movies by score
29 ranked_item_score = item_score.sort_values(by='movie_score', ascending=False)
30
31 # Select top m movies
32 m = 10
33 ranked_item_score.head(m)
```

10 rows × 2 columns	
movie	movie_score
16 Harry Potter and the Chamber of Secrets (2002)	1.888889
13 Eternal Sunshine of the Spotless Mind (2004)	1.888889
6 Bourne Identity, The (2002)	0.888889
29 Ocean's Eleven (2001)	0.888889
18 Inception (2010)	0.587491
3 Beautiful Mind, A (2001)	0.466667
5 Blade Runner (1982)	0.466667
12 Donnie Darko (2001)	0.466667
10 Departed, The (2006)	0.256727
31 Shawshank Redemption, The (1994)	0.222566

6. Popularity-Based Filtering (Extra)

Focusing on identifying trending content. Highlights of what is currently popular among users.

6.1 Overall Popularity

Rank movies by the total number of ratings they have received across the entire dataset.

```

1 # Group by movie title and count the total number of ratings
2 overall_popularity = mtrain_data.groupby('title')['Rating'].count().reset_index()
3
4 # Rename columns for clarity
5 overall_popularity.columns = ['Title', 'Num_Ratings']
6
7 # Sort by the number of ratings in descending order
8 overall_popularity = overall_popularity.sort_values(by='Num_Ratings', ascending=False)
9
10 # Display the top trending movies
11 overall_popularity.head(10)
✓ [466] 30ms

```

	Title	Num_Ratings
3158	Forrest Gump (1994)	329
7593	Shawshank Redemption, The (1994)	317
6865	Pulp Fiction (1994)	307
7680	Silence of the Lambs, The (1991)	279
5512	Matrix, The (1999)	278
8001	Star Wars: Episode IV - A New Hope (1977)	251
4662	Jurassic Park (1993)	238

6.2 Average Rating Poularity

Rank movies by their average rating instead of the count. This highlights the highest-rated movies.

```

1 # Group by movie title and calculate the average rating
2 average_rating_popularity = mtrain_data.groupby('title')['Rating'].mean().reset_index()
3
4 # Rename columns for clarity
5 average_rating_popularity.columns = ['Title', 'Avg_Rating']
6
7 # Sort by average rating in descending order
8 average_rating_popularity = average_rating_popularity.sort_values(by='Avg_Rating', ascending=False)
9
10 # Display the top movies by average rating
11 average_rating_popularity.head(10)
✓ [407] 28ms

```

	Title	Avg_Rating
3354	Gena the Crocodile (1969)	5.000000
8951	True Stories (1986)	5.000000
1986	Cosmic Scrat-tastrophe (2015)	5.000000
5269	Love and Pigeons (1985)	5.000000
7033	Red Sorghum (Hong gao liang) (1987)	5.000000
8661	Thin Line Between Love and Hate, A (1996)	5.000000
5019	Lesson Faust (1994)	5.000000